Topology-Aware Mapping for Photonic Interconnects in TPU-Class LLM Accelerators

Pranav Sitaraman
MIT 6.5930
sitaraman.pranav@gmail.com

Simone Surdi
MIT 6.5930
simonesurdi@gmail.com

***Abstract*—We evaluate Lightmatter-style photonic interconnect as an emerging technology for TPU-class accelerator pods running large language model training and inference. The interesting question is not just whether optical links lower the per-bit energy of a wire. It is whether their distance-insensitive transmission, hardware multicast, and high-radix optical switching change the memory placements and distributed schedules that a pod *should* use. To answer this we add a logical cluster-memory level above per-chip memory in an existing accelerator mapper, extract the mapper's traffic, price that traffic with a distance-aware network model, and feed the resulting energy and latency back into the memory hierarchy until the system stabilizes. The same spine then scales out from a 16-node cluster to a 64-chip torus building block and finally to a 9216-chip Ironwood-class pod, with a deliberately strong 3.5 Tb/s, 6 pJ/bit chiplet baseline included so the comparison is not against straw electronics. Across the study, the inner mapping inside each chip is largely invariant under the interconnect swap, but the distributed schedule above it changes: multicast cache fill, remote pooled-HBM streaming, and flatter topologies become rational. That distinction explains both the dramatic inference gains we report and the much more modest training gains.**

I. Introduction

THE latest generation of TPU pods is one of the clearest demonstrations that modern accelerator design is, more than anything, a data-movement problem. A single TPU7x/Ironwood pod connects 9216 chips with 192 GiB of HBM each and 1200 GB/s of bidirectional inter-chip bandwidth per chip, and it is deployed for both dense and mixture-of-experts language models [1]. For a hardware project, the practical question becomes less "can we make a faster wire?" than "which schedules suddenly become attractive when the wire stops being the bottleneck?"

Lightmatter's Passage family is a concrete and well-documented reference point for that thought experiment. The publicly disclosed M1000 product page reports 114.6 Tb/s of aggregate bidirectional bandwidth and 2.3 pJ/bit of effective system efficiency including the laser [3], while the EVK100 product page targets a 3.2 Tb/s per-fiber class point at 112G PAM4 [4], [5]. Photonics is interesting at pod scale not because the link is magically free, but because almost all of its energy is conversion and SerDes, and the transmission across fiber or waveguide is nearly distance-independent. Electrical links behave the opposite way: the conversion is cheap, but the transmission energy grows with the physical path the signal has to cross.

Our claim is that swapping in photonic interconnect is not a constant-factor link substitution. It is most useful when the system above it can adopt a different *distributed schedule*: replacing repeated unicast collectives with one logical multicast, replacing local-only weight residency with streaming from pooled remote HBM, and flattening hop-heavy mesh or torus topologies into single-hop fabrics. Concretely, the contributions of this paper are:

- an iterative network-aware feedback loop in which the mapper sees the topology cost as part of the memory hierarchy, not as a post-hoc adjustment;
- a distance-parameterized link model that separates conversion energy from per-millimeter transmission energy, so electrical, chiplet, and photonic links can be compared across cluster scales without collapsing them to a single pJ/bit number;
- a clear separation between the mapping the accelerator chooses locally (loop order, tile shape, residency) and the schedule the compiler chooses across the pod (sharded collectives, remote HBM, multicast cache fill); and
- a compact end-to-end evaluation, from a 16-node validation cluster up to an Ironwood-class 9216-chip pod, on public Gemma 4 model anchors and explicitly labeled long-context Gemini-class stress proxies.

II. Related Work and Course Context

Mapping and dataflow. In the course's vocabulary, an einsum defines the computation but not the order in which it executes. A mapping then chooses partitioning, temporal loop order, spatial loop order, tile sizes, and which tensor ranks are resident at which memory level. Recent work on accelerator mappers makes this distinction explicit: it argues that hardware comparisons are unreliable unless the mapper is given a real chance to find a good mapping [7]. That warning shapes our methodology directly. We do not report an interconnect comparison from a static spreadsheet. Instead, we let the mapper run, observe the traffic implied by its mapping, charge that traffic through the topology model, and rerun the mapper with the updated logical-memory cost.

Photonic accelerator modeling. Most prior photonic deep-learning modeling has focused inside the chip—photonic compute-in-memory elements that span electrical, analog, optical, and digital domains [8]. Our project asks the inverse:

hold the compute chip fixed and ask how the *inter-chip* fabric reshapes the system.

TPU systems and the compiler. TPU v5p and TPU7x both expose high-bandwidth ICI as a central system feature [1], [2]. XLA/GSPMD and PyTorch/XLA SPMD give the practical software analogy: the user describes how each tensor is sharded across a logical device mesh, and the compiler inserts the corresponding collective operations [6]. We do not implement an XLA pass. We model the relevant compiler-visible options explicitly—SPMD-style collectives, remote pooled-HBM streaming, replicated HBM caching, and photonic multicast cache fill—so we can ask which of them becomes optimal under each interconnect.

III. Methodology

Figure 1 summarizes the structure of the work. The model has four conceptual layers and we are careful to keep them separate.

Workload. Each LLM stage is represented as a parameterized GEMM-style einsum. Six representatives cover decode-time MLP, single- token KV attention, dense prefill projection, long-context prefill, mixture-of-experts dispatch, and a training-time weight-gradient computation.

Architecture. Above the per-chip memory hierarchy (MAC, register file, unified buffer, HBM) we add a single logical cluster memory representing the union of accelerator memories in a multi-chip system. The mapper is allowed to spill or stream tensors to that level just as it would to off-chip DRAM. Its energy and latency parameters are exactly what the iterative loop will update.

Inner mapping. Given the architecture and the einsum, the mapper emits a mapping: storage, temporal, and spatial loop nodes plus the compute-node tile shape and residency. We fingerprint this mapping into a compact signature, both to detect convergence and to ask whether it actually changes when we change the interconnect underneath.

Distributed schedule. Above the inner mapping, a separate choice decides whether the cross-chip traffic is node-to-node, fetched from a shared remote DRAM, or multicast from a shared source to many nodes. This is where the XLA/GSPMD analogy lives. The point is that this layer can change while the inner mapping stays the same, and one of our main results is that this is exactly what happens.

A. Network and Distance Model

For a transfer of B bits across an average hop count h, with average per-hop physical distance $\ell(N)$, link bandwidth BW, and contention factor $C(N)$ on N active nodes, the network energy and latency are

$$E_{\text{net}} = B\big(e_c + h\,\ell(N)\,e_{tx}\big),$$

$$T_{\text{net}} = \frac{B}{BW/C(N)} + h\,t_r + h\,\ell(N)\,t_p + t_s,$$

where e_c is the conversion energy per bit, e_{tx} is the per-millimeter transmission energy, t_r is per-hop router or fanout-element delay, t_p is propagation delay, and t_s captures optical circuit-switching reconfiguration or packet-switching arbitration. The crucial split is the e_c versus e_{tx} term: electrical and chiplet baselines pay an e_{tx} that grows with the size of the system, while photonic links are conversion-dominated and remain nearly flat as the path grows. Multicast-capable optical fabrics pay the unique payload exactly once; non-multicast baselines pay it once per receiver.

TABLE I: Compact assumptions used throughout the study. Where Google does not publish a number, the assumption is labeled and the model is run as a sensitivity sweep around that point.

Item	Value and role
Ironwood pod	9216 chips, 192 GiB HBM/chip, 1200 GB/s bidirectional ICI/chip — defines the full-system scale
M1000 photonic	57.3 Tb/s one-way, 2.3 pJ/bit including laser, multicast-capable optical circuit switch
EVK100 photonic	3.2 Tb/s per-fiber class, 1.9 pJ/bit reference point with a 3.2 pJ/bit conservative point kept for sensitivity
Electrical link	0.2 pJ/bit conversion plus 1.96 pJ/bit/mm transmission, calibrated to a short-reach reference
Photonic link	2.3–4.5 pJ/bit conversion plus near-zero per-mm transmission
Chiplet baseline	3.5 Tb/s, 6 pJ/bit/hop, deliberately strong non-optical comparison
Compiler layer	SPMD collectives, remote pooled-HBM streaming, replicated cache, photonic multicast

B. Iterative Network-Aware Feedback Loop

The mapper is run with the logical memory level initialized to plain HBM cost. Its emitted action counts are converted to bits and split by tensor; the topology is then asked for the corresponding read and write energy and latency under the active scenario; the logical-memory level's energy and latency are updated to include those network terms; and the mapper is rerun. The loop stops when the per-bit numbers stabilize. Because the loop modifies the mapper's inputs rather than just its outputs, the mapper has the chance to choose a different mapping in response to the network cost. As we show below, in our settings it mostly chooses to keep the same inner mapping, even though the distributed schedule above it does change.

IV. Results

A. Distance Physics and Scaling

Figure 2 sets up the physical reasoning behind every later result. At very short reach, an electrical link can win on raw pJ/bit because it skips the optical conversion. As the system grows, the electrical and chiplet curves accumulate transmission energy linearly with distance; the photonic curve does not. A regression test in the project's automated suite locks the crossover in place, so a later parameter tweak cannot silently invert the physics.

The right panel separates strong scaling (a fixed amount of work spread over more chips) from weak scaling (work per chip held roughly constant while the pod grows). Photonics is strongest when the same shared state is needed by many chips—decode caches, expert weights, or layer-tied parameters—because that is exactly what multicast collapses. Workloads

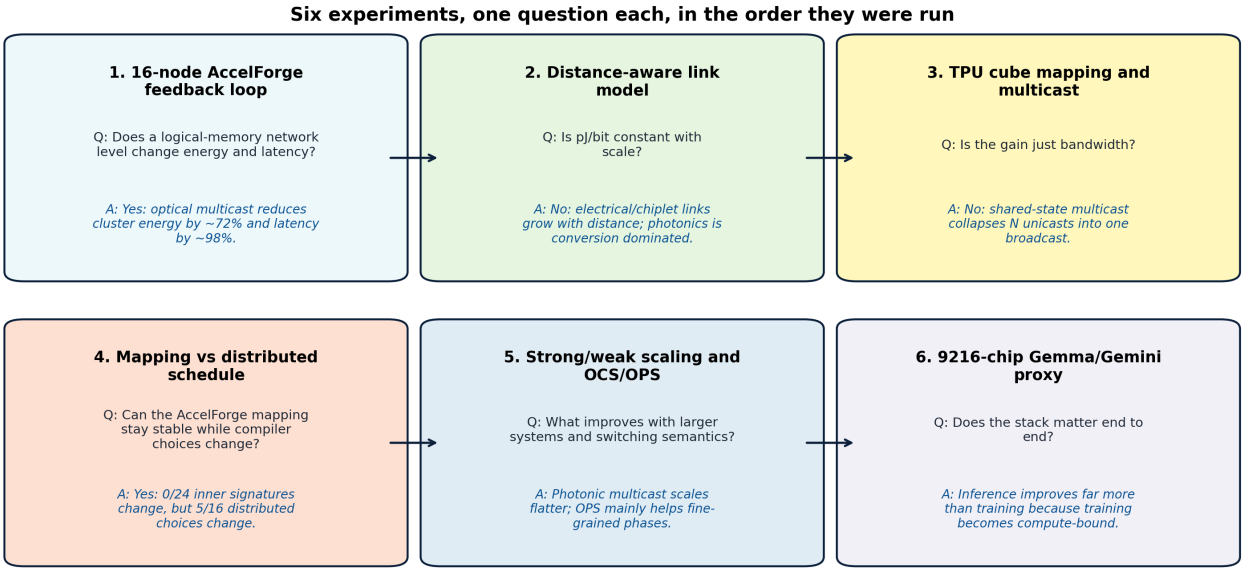


Fig. 1: The study is laid out as one evidence chain. Each experiment exists because the previous one leaves a specific question unanswered, from logical-memory feedback to distance physics, multicast, mapping versus schedule, scaling, and the end-to-end Ironwood-class pod model.

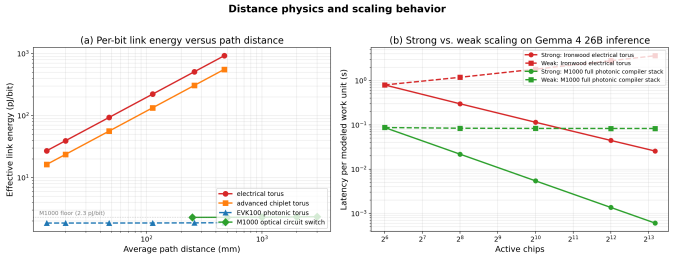


Fig. 2: Per-bit link energy versus average path distance and strong/weak scaling on a Gemma 4 26B inference workload. Electrical torus and advanced chiplet links pay an energy term that grows with distance. Photonic torus and switched fabric links are dominated by conversion energy, so they remain effectively flat across the cluster range.

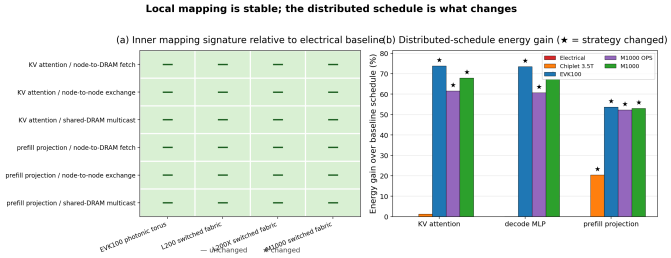


Fig. 3: Inner mapping signatures versus distributed-schedule decisions on the 64-chip cube. Across all photonic variants and tested workloads the inner mapping is unchanged from the electrical baseline (panel (a)), while the compiler-visible schedule layer above it does shift in favor of multicast or remote pooled-HBM streaming and gains tens of percent in energy (panel (b)).

that do not exercise that pattern see less benefit, and we keep both modes visible to avoid conflating them.

B. Where Mapping Actually Changes

Figure 3 is the central modeling result of the paper. The left matrix shows that on the 64-chip torus the local mapping signature is unchanged across every photonic variant we tested, for every modeled traffic case. That is not the usual narrative for an "emerging interconnect" study, but it is exactly what one should expect once the experiment is set up cleanly: the local GEMM dataflow, tile size, and residency choices are dominated by per-chip memory and compute parameters, which we hold fixed.

What does change is the schedule one layer above. The right panel shows that the compiler-visible energy gain across workloads is large for photonic technologies and ranges from roughly half to nearly three quarters of the original schedule energy, while it is essentially zero for the electrical baseline because the alternative schedules cost more under that link. The strategies that win are remote pooled-HBM streaming for decode and KV attention, and shared-DRAM multicast for prefill projection. This is the layer we mean when we say "distributed schedule"—it is what the GSPMD-style compiler would choose differently if the underlying interconnect changed.

C. Multicast and a Strong Chiplet Baseline

The chiplet baseline in Figure 4 is intentionally strong: 3.5 Tb/s per link and 6 pJ/bit per hop, similar to what advanced packaging is targeting. We use it specifically to defeat the hypothesis that the photonic advantage is just a bandwidth gap. The left panel pulls the absolute energy of every cube

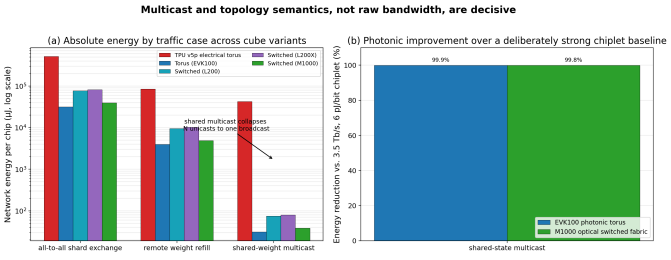


Fig. 4: Per-chip network energy by traffic case across cube variants on a logarithmic axis (panel (a)) and energy reduction over a deliberately strong 3.5 Tb/s, 6 pJ/bit chiplet baseline on shared multicast traffic (panel (b)). Photonics still wins by roughly two orders of magnitude on shared multicast, because the topology semantics change rather than just the bandwidth.

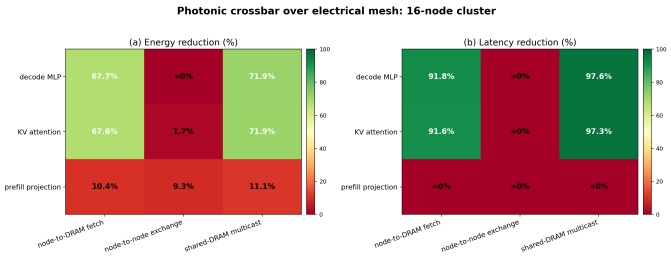


Fig. 5: Energy and latency reduction of the 16-node photonic crossbar relative to the electrical mesh, on the validation cluster used in the feedback loop.

variant onto a logarithmic axis. The shard-exchange and remote-weight-refill cases sit roughly an order of magnitude below the electrical torus, but the shared-weight multicast case sits nearly four orders of magnitude below it, because the optical fabric only pays the unique payload once. The right panel restates the chiplet comparison: even relative to that deliberately stronger baseline, photonic torus and switched fabric options reduce shared-state multicast energy by close to 100 %. Figure 5 then shows the same effect on the small 16-node cluster used to validate the feedback loop. Energy reductions of roughly 68–72 % on remote weight fetch and shared multicast, with corresponding latency reductions above 90 %, are stable across weight-dominated workloads.

D. End-to-End Pod Impact

Figure 6 pushes the full stack to a 9216-chip Ironwood-class pod. We use the public Gemma 4 26B mixture-of-experts and 31B dense open-model anchors for sizing and prompt and output token counts, and a long-context Gemini-class stress proxy whose parameter count Google does not publish; we therefore label it as an explicit proxy rather than a Gemini measurement. With the full M1000 photonic compiler stack engaged, the modeled inference latency speedup over the electrical baseline is roughly $42\times$ for the Gemma 4 MoE proxy and $15\times$ for the long-context Gemini proxy. Training speedups are much smaller—around $3.4\times$ and $1.4\times$ respectively—and the right panel of the figure makes the reason concrete: on the electrical baseline, more than 95 % of the inference latency

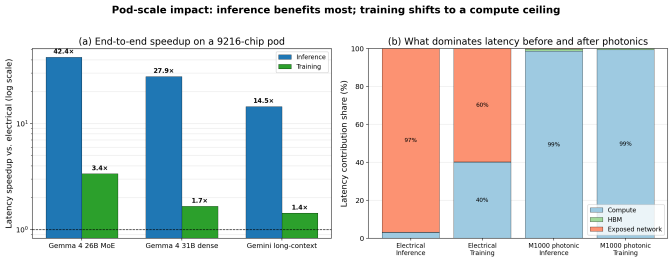


Fig. 6: End-to-end speedup on a 9216-chip Ironwood-class pod (panel (a), log scale) and the latency contribution share of compute, HBM, and exposed network before and after photonics (panel (b)). Inference shifts from being almost entirely network-bound on the electrical baseline to almost entirely compute-bound under the photonic stack; training is more compute-balanced to begin with, so its remaining improvement is smaller.

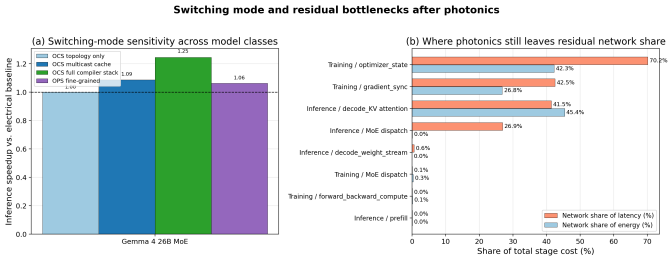


Fig. 7: Inference speedup across switching modes for three model classes (panel (a)) and residual network share of latency and energy broken down by stage on the Gemma 4 26B MoE proxy after the photonic stack is enabled (panel (b)).

lives in exposed network. Photonics removes that share, which is what the $42\times$ figure represents. Once it is removed, what is left is matrix-multiply compute. The training comparison starts from a more balanced split, and so improves less in relative terms.

E. Switching Mode and Residual Bottlenecks

Figure 7 unpacks the photonic stack itself. A topology-only optical circuit switch already brings the system to parity with electrical at the modeled scale; layering multicast cache fill on top adds an extra $1.1\times$; running the full compiler stack then takes the inference speedup to between $1.06\times$ and $1.25\times$ across the three model classes (relative to the electrical baseline of $1.0\times$). A fine-grained optical packet-switching sensitivity point, shown alongside the OCS modes, is included to argue that switching mode itself is not where most of the gain comes from—the gain comes from the combination of multicast and topology, not from packet switching specifically.

The right panel makes the residual bottleneck explicit. After photonics, the largest remaining network share lives in the training optimizer-state and gradient-synchronization stages, where 30–70 % of the latency and 25–45 % of the energy still belong to the network. Inference KV attention and MoE dispatch retain a smaller but non-zero residual network share. Decode weight streaming and forward compute drop almost completely out of the network category once the photonic stack is engaged.

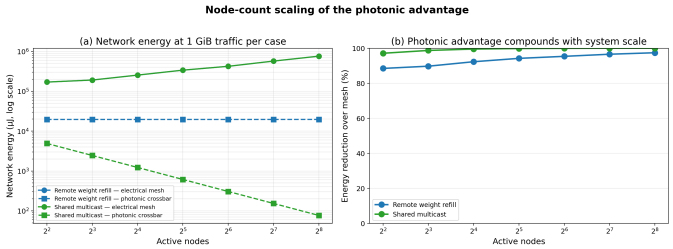


Fig. 8: Pure-network scaling sweep: the photonic advantage compounds as the number of active nodes grows, because the electrical mesh's average path length and contention both increase while the optical fabric stays single-hop and uncontended.

TABLE II: Headline results across the four levels of the study.

Layer	Best result	Interpretation
16-node cluster	71.95 % energy, 97.57 % latency reduction	The logical-memory feedback loop changes system metrics, and the optical multicast advantage shows up immediately
64-chip cube network	99.93 % energy reduction	Multicast collapses repeated unicasts
64-chip cube mapping	0 of 24 inner mappings change	Local dataflow is dominated by per-chip parameters
Distributed schedule	5 of 16 schedule choices change	Compiler-visible memory placement is where the new paradigm appears
Compiler strategy	73.86 % best energy gain	Remote pooled-HBM and multicast cache fill can be optimal
9216-chip inference	42.43× speedup, 94.05 % energy reduction (Gemma 4 MoE)	Decode and expert routing benefit most
9216-chip training	3.37× speedup (Gemma 4 MoE)	Communication shrinks, then compute dominates

Figure 8 confirms the same trend in a cleaner analytical sweep. As the active node count grows from 4 to 256, the electrical mesh's energy on remote weight fetches and shared multicast grows because both the average path length and the contention factor grow with the system, while the photonic crossbar remains effectively flat. The advantage compounds: at 256 nodes, more than 99 % of the network energy on shared multicast can be removed.

V. Discussion and Limitations

The strongest claim in this paper is conditional. Photonic interconnect is most valuable when the system above it actually exposes multicast, pooled memory, and topology choice to the compiler. Workloads that have very little inter-chip traffic, that run over short electrical paths, or that are already compute-dominated do not move very much. We deliberately included those cases—node-to-node prefill exchange on the 16-node cluster, training-side speedups on the pod model, and the topology-only switching mode—because the negative results are part of the picture.

A second observation worth stating clearly is the mapping versus schedule split. The fact that the local mapping does not move under the interconnect swap is not a weakness of the experiment. It is what allows the compiler-visible schedule changes to be attributed cleanly to the network rather than to a coincidental retiling. We therefore report the inner-mapping signatures and the distributed-schedule choices as separate columns rather than collapsing them into a single "mapping changed" number.

Finally, the model has explicit scope. It includes mapper action counts, per-chip memory cost, distance-aware link energy, hop count, contention, multicast volume, optical-switching sensitivity, and a stage-level overlap term that prevents double-counting compute and network time. It does not include packet-level queueing, thermal drift across the photonic stack, laser aging, area or cost analysis, model quality effects, or the specific code generation paths a production XLA compiler would take. Each of these is a candidate for follow-on work rather than a hidden assumption.

VI. Conclusions

Across a 16-node cluster, a 64-chip torus, and a 9216-chip pod, the clearest picture is consistent. Photonic interconnect is not a universal multiplier on top of an existing accelerator. It is the right answer when the application has shared state to broadcast, when the topology is large enough that distance starts to dominate electrical link energy, and when the compiler is allowed to choose schedules—multicast cache fill, pooled remote HBM, flatter fabrics, high-radix switching—that are simply infeasible at the same cost on an electrical baseline. With those conditions met, the inference side of an Ironwood-class pod becomes compute-bound rather than network-bound, which is the regime where photonics earns its place.

References

[1] Google Cloud, "TPU7x (Ironwood)," 2026. [Online]. Available: https://docs.cloud.google.com/tpu/docs/tpu7x

[2] Google Cloud, "Cloud TPU v5p," 2026. [Online]. Available: https://cloud.google.com/tpu/docs/v5p

[3] Lightmatter, "Passage M1000 EVK," 2026. [Online]. Available: https://lightmatter.co/products/passage-m1000-evk/

[4] Lightmatter, "Passage EVK100," 2026. [Online]. Available: https://lightmatter.co/products/passage-evk100/

[5] Lightmatter, "Passage," 2026. [Online]. Available: https://lightmatter.co/products/passage/

[6] PyTorch/XLA, "SPMD User Guide," 2026. [Online]. Available: https://docs.pytorch.org/xla/master/spmd.html

[7] M. Gilbert, T. Andrulis, V. Sze, and J. Emer, "The Turbo-Charged Mapper: Fast and Optimal Mapping for Accelerator Modeling and Evaluation," arXiv:2602.15172, 2026.

[8] S. Zheng *et al.*, "CiMLoop for Photonics," MIT 6.5930 advanced-technology material, 2026.

[9] Google, "Gemma 4," 2026. [Online]. Available: https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/